Lecture 5

Fernando Birra
Rui Nóbrega

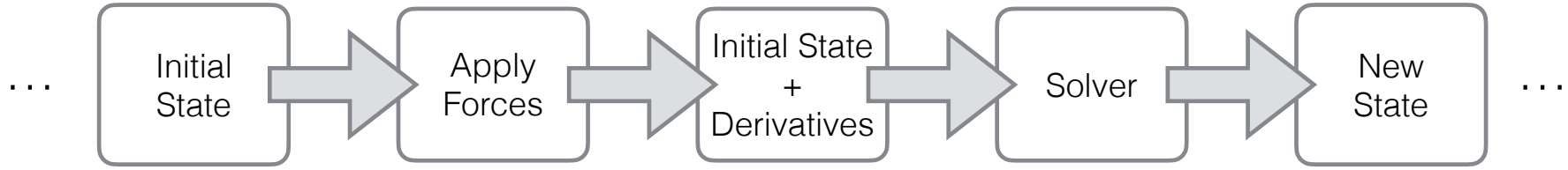
Simulation

Fundamentals

Fernando Birra
Rui Nóbrega

Physics-Based Animation

- Applications ranging from particles to rigid bodies
- Moving from Kinematics (animator describes $\mathbf{x}$) to dynamics (computer solves $\mathbf{f} = m\mathbf{a}$, to find $\mathbf{x}(t)$)



The Simulation Loop (simplified)

- Repeat for all particles/bodies:

  1. **Clear Forces**: $\mathbf{f}_i = \overrightarrow{0}$
  2. **Compute and Apply Forces**: $\mathbf{f}_i += \ldots$ (gravity, springs, wind, user interaction, …)
  3. **Integrate**: Compute new state $\mathbf{S}$ ($\mathbf{x}_{i+1}$ and $\mathbf{v}_{i+1}$), given current state $\mathbf{S}$ ($\mathbf{x}_i$ and $\mathbf{v}_i$), its time derivatives $\dot{\mathbf{S}}$ ($\mathbf{v}_i$ and $\mathbf{a}_i$), and Δt.
  4. **Detect Collisions** (Check for superposition)
  5. **Resolve Collisions** (Apply impulses)
  6. Update Scene

- Adjusting Δt is critical for stability, as well as the method chosen for step 3.

Accumulating Forces

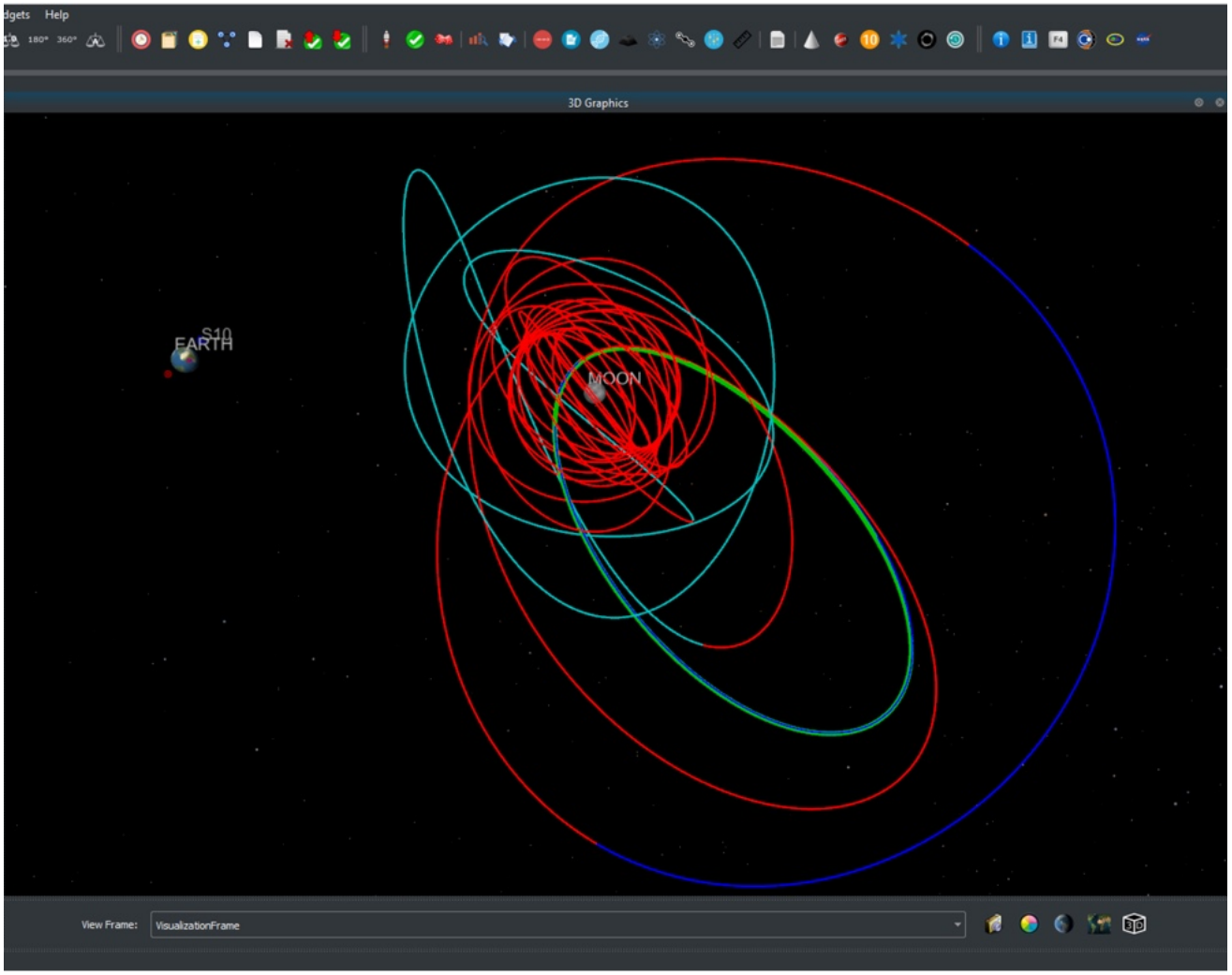
- Individual forces acting on a particle are added together to get the resultant force.
- **Contact Forces**:
  - Normal
  - Friction (static and Kinetic)
  - Tension
  - Spring
  - Drag/Air Resistance
  - External
- **Field or Non-Contact Forces**:
  - Gravitational
  - Electric
  - Magnetic
  - Electromagnetic

Particle Systems

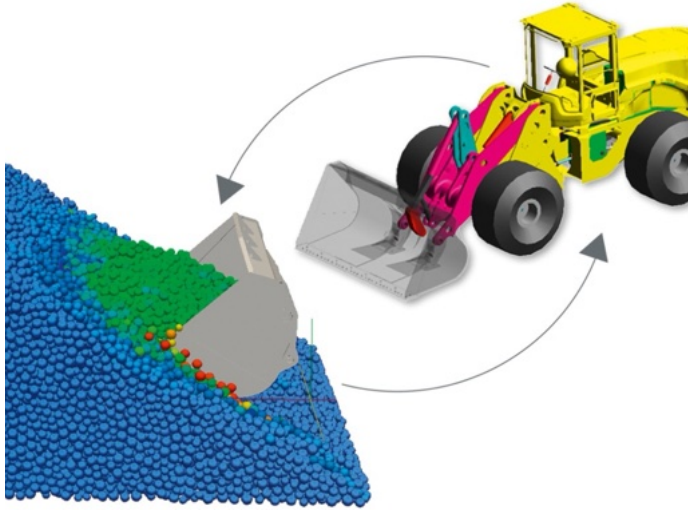
Fernando Birra
Rui Nóbrega

Some history…

- Important tool for Scientific Simulation in Chemistry, Physics, Engineering etc.
- Introduced in the 50s by Alder and Wainwright with the initial hard spheres model for liquid simulation.
- In 1964, Rahman extended particles to simulate liquid argon properties.
- The term Particle Systems was coined by William Reeves at Lucasfilm in 1983, while working for the Movie Star Trek II - The Wrath Of Khan.
- In 1986, Craig Reynolds extended particle systems to simulate Flocks, Herds and Schools using what he called boids.
- 1980/1990 witnessed the use of Smoothed Particle Hydrodynamics for realist flow of water and splashes, adapting from its original purpose in Astrophysics.

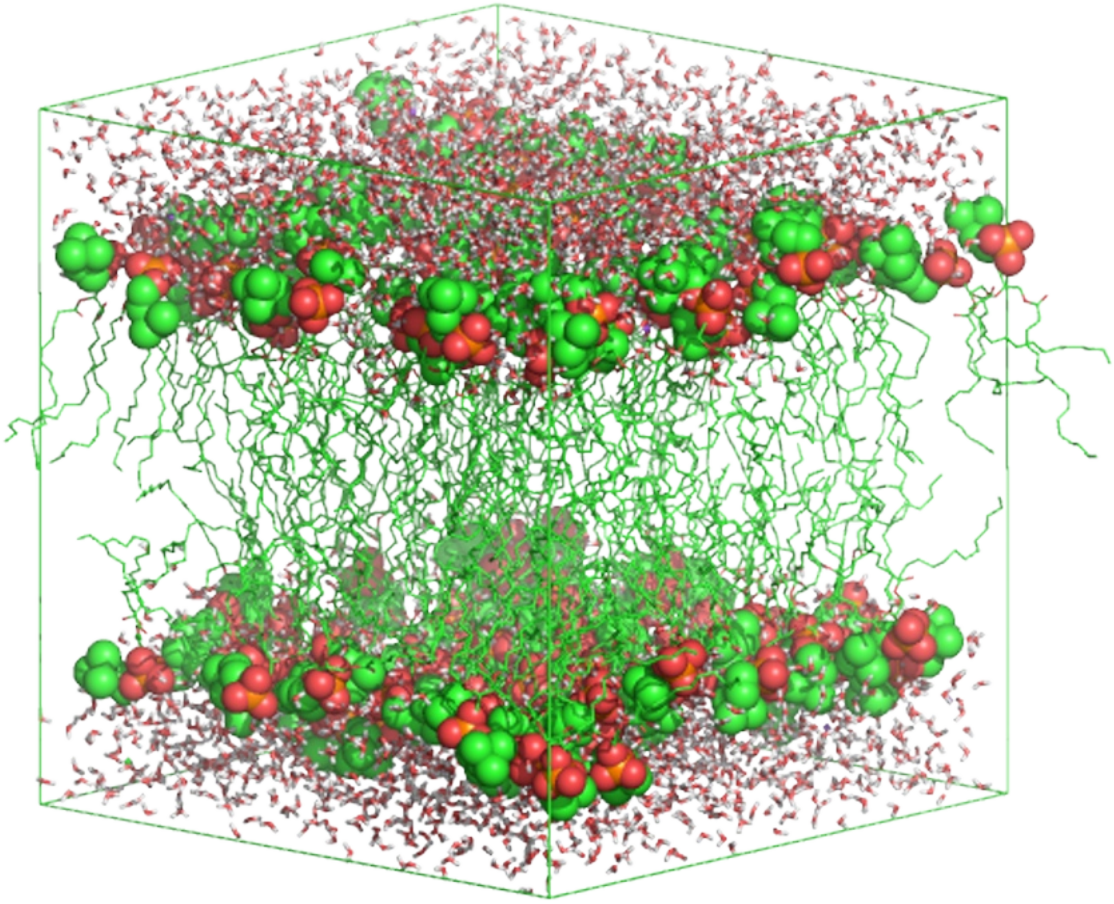


[Copernicus Trajectory Design, NASA]



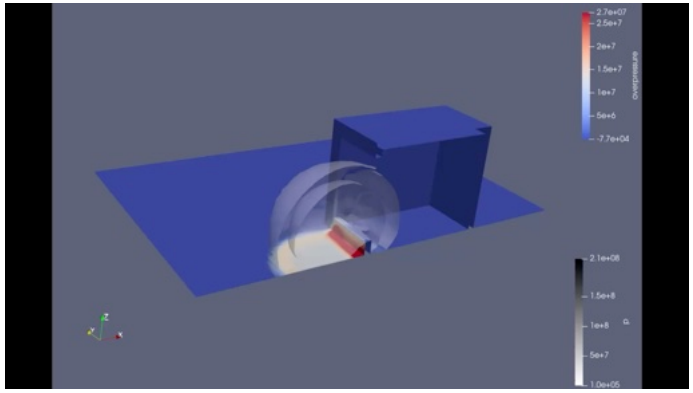
[Soil simulation, Fraunhoffer, 2018]

[Particle Simulation, Laura Remler]

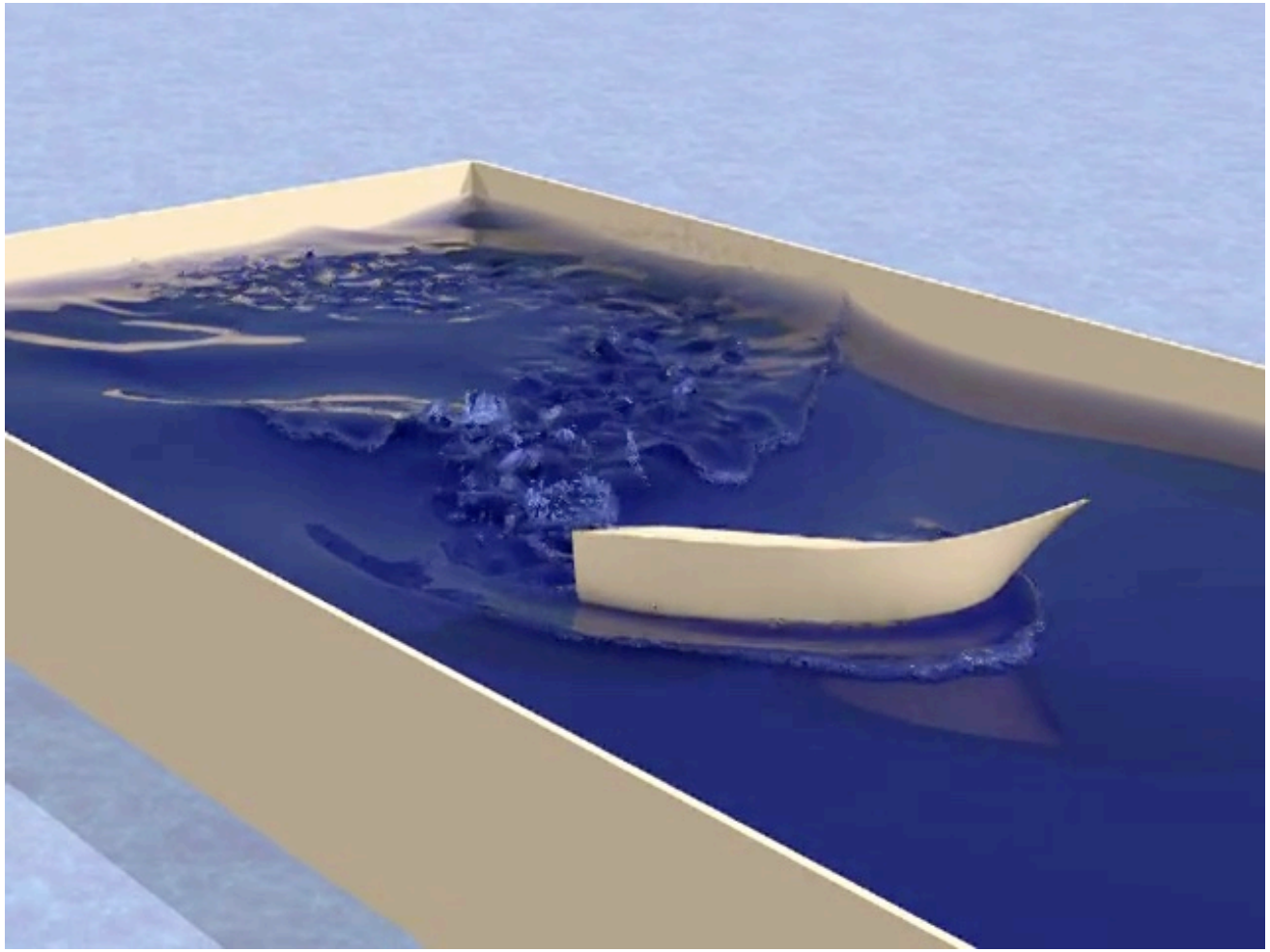


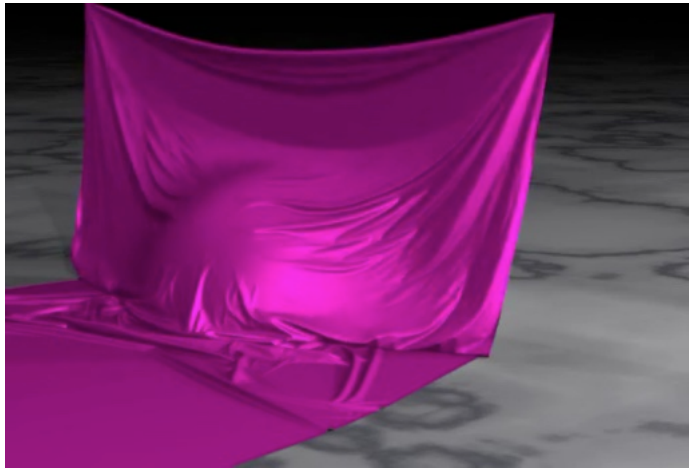
[Lipid bilayer simulation, Aponte-Santamaría, 2012]

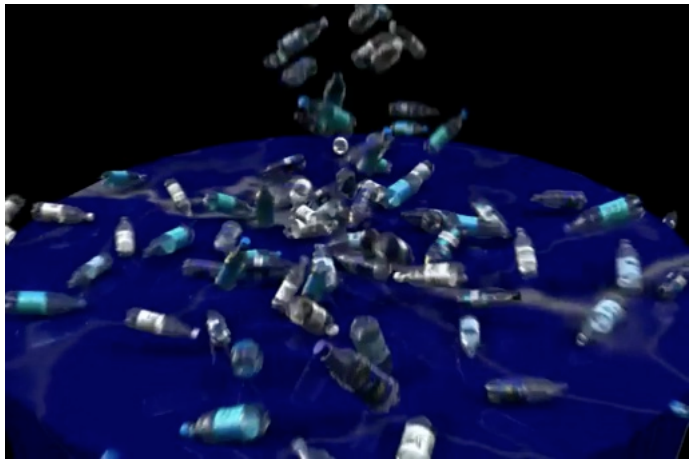
[Kerbal Space Program]



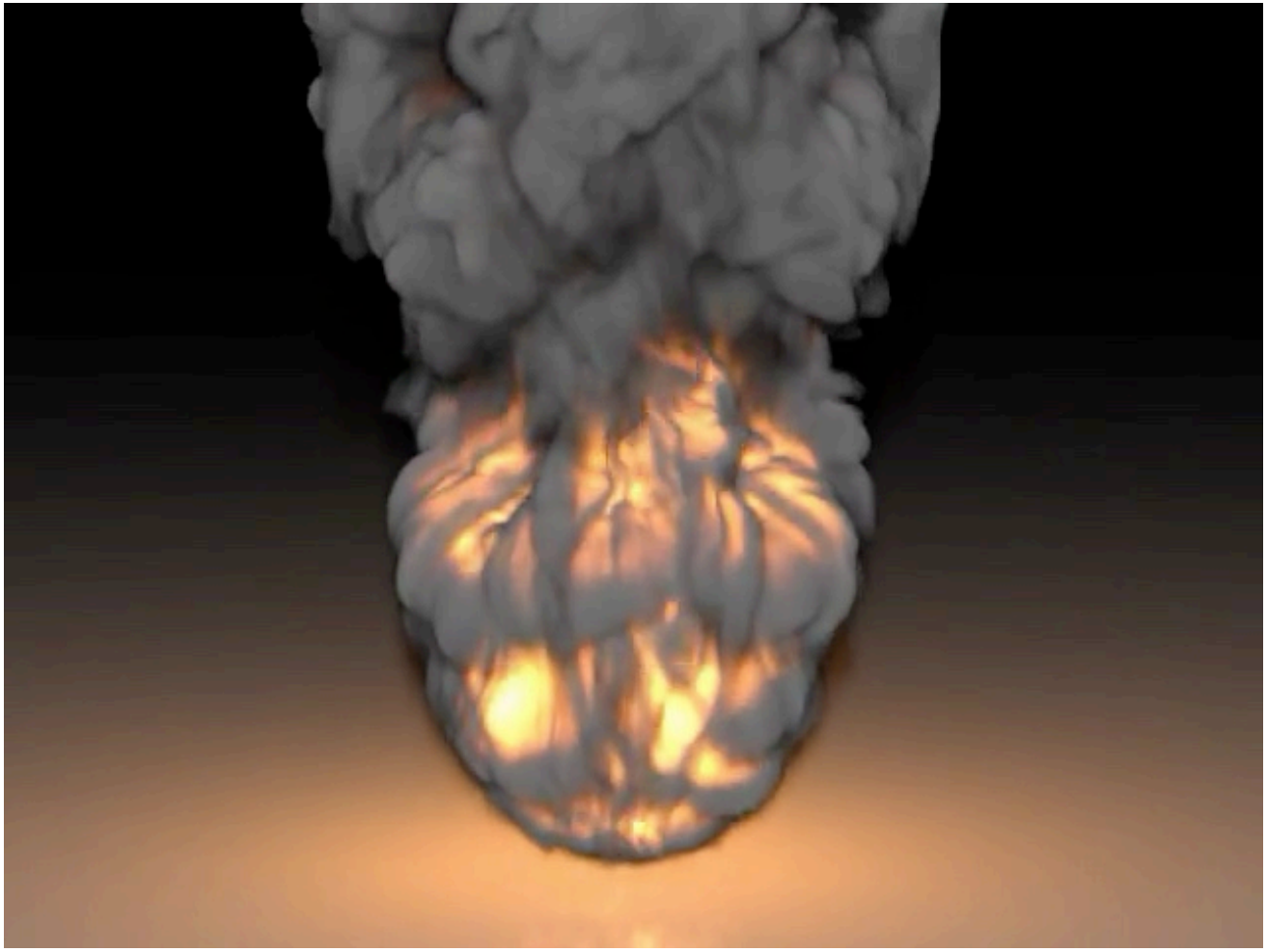
[Blasfoam + Paraview]







Fernando Birra
Rui Nóbrega



[Fluid Dynamics Simulations]

Newtonian Particle P_i

- Each particle has:
  - mass m_i and position $\mathbf{x}_i$.
  - Newton's 2nd law: $\dfrac{d^2\mathbf{x}_i}{dt^2} = \dfrac{\mathbf{f}_i}{m_i}$ needs to be solved for $\mathbf{x}_i$ to find particle trajectory over time.
  - This 2nd order ODE (Ordinary Differential Equation) can be transformed into a set of 2 1st order ODE, by introducing an auxiliary variable $\mathbf{v}_i$:

$$\begin{cases} d\mathbf{x}_i/dt = \mathbf{v}_i \\ d\mathbf{v}_i/dt = \mathbf{f}_i/m_i \end{cases}$$

Particle State Vector

$$\mathbf{S} = \begin{bmatrix} \mathbf{x}_i \\ \mathbf{v}_i \\ m_i \\ \mathbf{f}_i \end{bmatrix}$$ represents the state of a particle.

If all particles have the same mass, a gobal simulation variable m is enough

$$\dot{\mathbf{S}} = \frac{d}{dt}\mathbf{S} = \frac{d}{dt}\begin{bmatrix} \mathbf{x}_i \\ \mathbf{v}_i \\ m_i \\ \mathbf{f}_i \end{bmatrix} = \begin{bmatrix} \mathbf{v}_i \\ \mathbf{f}_i/m_i \\ 0 \\ \mathbf{0} \end{bmatrix}$$, represents the state's time derivative.

The simulation requires the ability to compute the forces that affect the particle.

This formulation allows us to treat any physics problem as an Ordinary Differential Equation (ODE).

Particle System S

$$S = \{particles, t\}$$

time

$$\dot{S} = dS/dt$$

$$\begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_t \qquad \begin{bmatrix} \mathbf{v}_1 & \mathbf{f}_1/m_1 & 0 & \mathbf{0} \\ \mathbf{v}_2 & \mathbf{f}_2/m_2 & 0 & \mathbf{0} \\ \mathbf{v}_3 & \mathbf{f}_3/m_3 & 0 & \mathbf{0} \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{v}_n & \mathbf{f}_n/m_n & 0 & \mathbf{0} \end{bmatrix}_t$$

Note: Represented data structures are for illustration purposes. Actual implementations vary and are more efficient.

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}
simulation_step(System S, float t, float Δt)
{




}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}
simulation_step(System S, float t, float Δt)
{
  clear_forces(S);



}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}

simulation_step(System S, float t, float Δt)
{
  clear_forces(S);



}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{0} \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{0} \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{0} \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{0} \end{bmatrix}_{t_0}$$

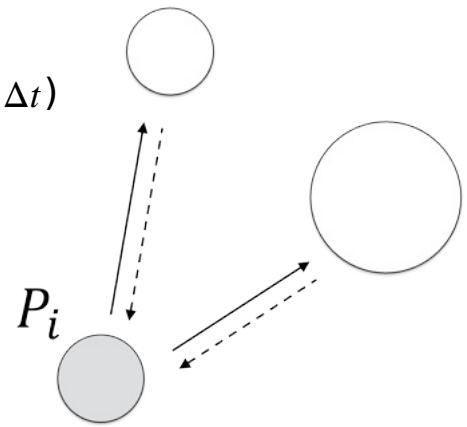
Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}

simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);


}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{0} \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{0} \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{0} \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{0} \end{bmatrix}_{t_0}$$



Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}

simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);


}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}

simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);
  dSdT = compute_state_derivative(S, t);

}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}
simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);
  dSdT = compute_state_derivative(S, t);

}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

$$\dot{S} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{a}_1 & 0 & \mathbf{0} \\ \mathbf{v}_2 & \mathbf{a}_2 & 0 & \mathbf{0} \\ \mathbf{v}_3 & \mathbf{a}_3 & 0 & \mathbf{0} \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{v}_n & \mathbf{a}_n & 0 & \mathbf{0} \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}
simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);
  dSdT = compute_state_derivative(S, t);
  update_state(S, dSdT);
}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0}$$

$$\dot{S} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{a}_1 & 0 & \mathbf{0} \\ \mathbf{v}_2 & \mathbf{a}_2 & 0 & \mathbf{0} \\ \mathbf{v}_3 & \mathbf{a}_3 & 0 & \mathbf{0} \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{v}_n & \mathbf{a}_n & 0 & \mathbf{0} \end{bmatrix}_{t_0}$$

Particle Systems Simulation

```
simulation(System S, float t0, float tf, float Δt)
{
  t = t0;
  while(t < tf) {
    simulation_step(S, t, Δt);
    t += Δt
    // Render system
  }
}
simulation_step(System S, float t, float Δt)
{
  clear_forces(S);
  apply_forces(S, t);
  dSdT = compute_state_derivative(S, t);
  update_state(S, dSdT);
}
```

$$S = \begin{bmatrix} \mathbf{x}_1 & \mathbf{v}_1 & m_1 & \mathbf{f}_1 \\ \mathbf{x}_2 & \mathbf{v}_2 & m_2 & \mathbf{f}_2 \\ \mathbf{x}_3 & \mathbf{v}_3 & m_3 & \mathbf{f}_3 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{x}_n & \mathbf{v}_n & m_n & \mathbf{f}_n \end{bmatrix}_{t_0+\Delta t}$$

$$\dot{S} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{a}_1 & 0 & 0 \\ \mathbf{v}_2 & \mathbf{a}_2 & 0 & 0 \\ \mathbf{v}_3 & \mathbf{a}_3 & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots \\ \mathbf{v}_n & \mathbf{a}_n & 0 & 0 \end{bmatrix}_{t_0}$$

Integration Methods

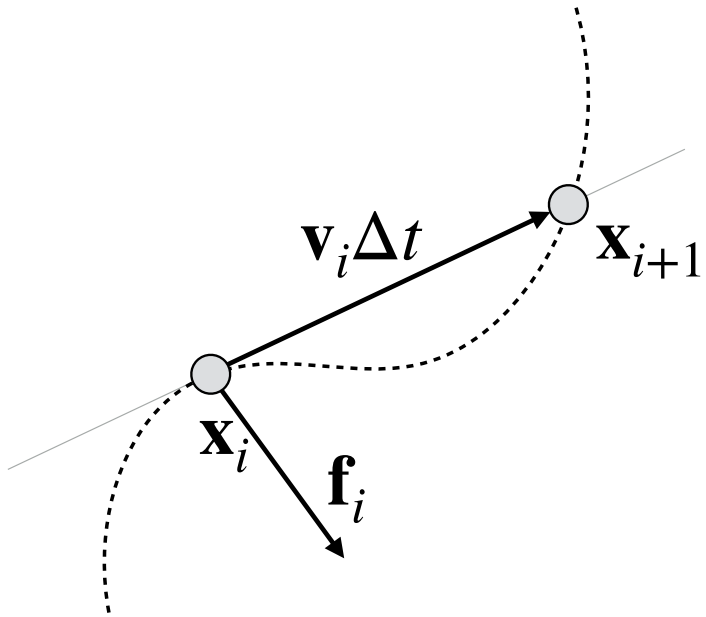
Fernando Birra
Rui Nóbrega

Numerical Integration

- $\mathbf{a} = \mathbf{F}/m$, but we need $\mathbf{x}$. We must “step” through time.
- The risk: Numerical instability

Explicit Euler

- First order method:
- $$\begin{cases} \mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n \Delta t \\ \mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t \end{cases}$$
- Alternative (second order Euler):
- $$\begin{cases} \mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n \Delta t + \frac{1}{2}\mathbf{a}_n \Delta t^2 \\ \mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t \end{cases}$$
- Simple but dangerous for physics
- Need very small time steps to reduce drift from real trajectories.
- Does not conserve energy!



1st Order Explicit Euler Step

Explicit Euler and Taylor Series

- $f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2!}f''(x_0) + \ldots + \frac{h^n}{n!}f^{(n)}(x_0)$
- Explicit Euler is equivalent to truncating by the second term (first order) or by the third term (second order).
- Considering position as the function to be integrated, we normally don't have access to derivatives of order ≥ 3.

Verlet Integration

- Math: $\mathbf{x}_{n+1} = 2\mathbf{x}_n - \mathbf{x}_{n-1} + \mathbf{a}\Delta t^2$

- Derived from adding the Taylor expansions for $f(x_0 + h)$ and $f(x_0 - h)$

$$
\begin{aligned}
& f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2!}f''(x_0) + \frac{h^3}{3!}f'''(x_0) + \epsilon(h^4) \\
+ \quad & f(x_0 - h) = f(x_0) - hf'(x_0) + \frac{h^2}{2!}f''(x_0) - \frac{h^3}{3!}f'''(x_0) + \epsilon(h^4) \\
\hline
& f(x_0 - h) + f(x_0 + h) = 2f(x_0) + h^2 f''(x_0) \Leftrightarrow f(x_0 + h) = \underline{2f(x_0) - f(x_0 - h) + h^2 f''(x_0)}
\end{aligned}
$$

- Why use it? Implicitly handles velocity and is remarkably stable for constrained systems (like cloth)

- However, it requires storing the last two states!

Verlet Velocity Integration

- Verlet Integration does not explicitly track velocity, which is needed sometimes (kinetic energy, drag, friction, etc.)

- Steps:

  1. Compute velocity at half-step: $\mathbf{v}_{n+1/2} = \mathbf{v}_n + \frac{1}{2}\mathbf{a}_n \Delta t^2$

  2. Compute next position (using velocity at half-step): $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n \Delta t + \frac{1}{2}\mathbf{a}_n \Delta t^2 = \mathbf{x}_n + \Delta t \mathbf{v}_{n+\frac{1}{2}}$

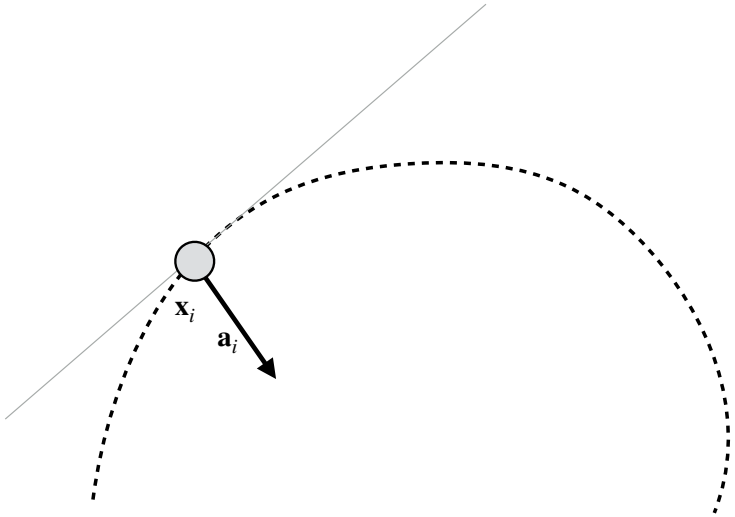
  3. Calculate new acceleration: $\mathbf{a}_{n+1}$ using the new position $\mathbf{x}_{n+1}$ (apply forces)

  4. Finish velocity update: $\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \frac{1}{2}\mathbf{a}_{n+1} \Delta t$

Verlet Velocity Integration

- Advantages:
  - **Stability**: Unlike Euler integration, where errors accumulate and "explode" (causing objects to fly off to infinity), Verlet tends to oscillate around the correct energy level.
  - (Pseudo)**Simplicity**: The basic version doesn't require storing velocities (but requires twice the positions).
  - **Constraints**: It is excellent for "Procedural Animation" (like cloth or ragdoll physics) because you can easily move positions to satisfy constraints and the velocity will "correct" itself in the next frame.

Midpoint or Runge-Kutta 2 (RK2)

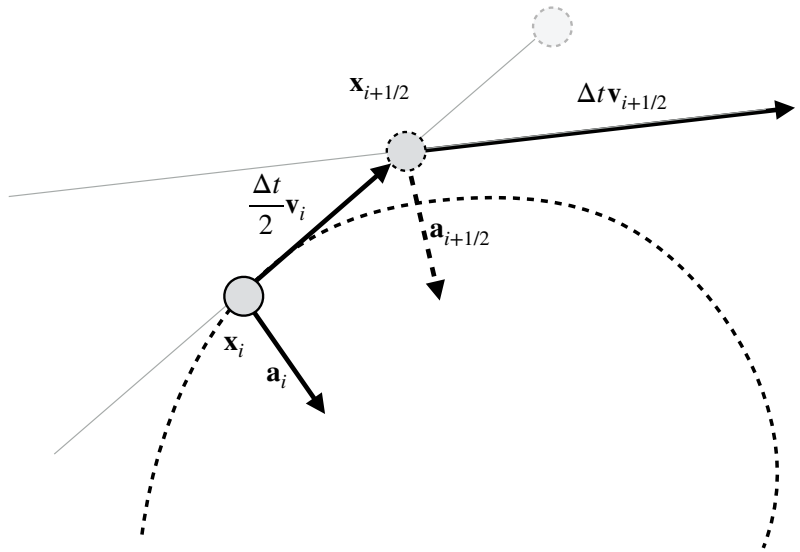


1st Order Explicit Euler Step

Midpoint or Runge-Kutta 2 (RK2)

- Perform an half-step to find intermediate values:

$$\begin{cases} \mathbf{x}_{i+\frac{1}{2}} = \mathbf{x}_i + \frac{\Delta t}{2}\mathbf{v}_i \\ \mathbf{v}_{i+\frac{1}{2}} = \mathbf{v}_i + \frac{\Delta t}{2}\mathbf{a}_i \end{cases}$$



1st Order Explicit Euler Step

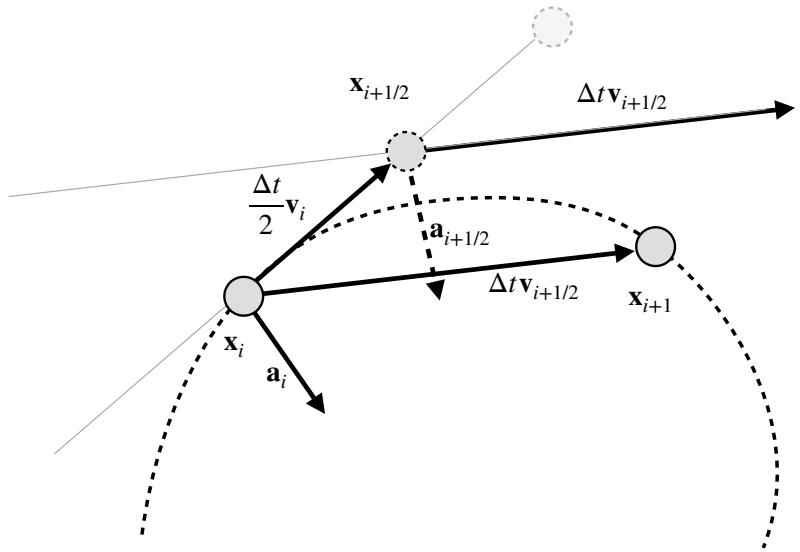
Midpoint or Runge-Kutta 2 (RK2)

- Perform an half-step to find intermediate values:

$$\begin{cases} \mathbf{x}_{i+\frac{1}{2}} = \mathbf{x}_i + \frac{\Delta t}{2}\mathbf{v}_i \\ \mathbf{v}_{i+\frac{1}{2}} = \mathbf{v}_i + \frac{\Delta t}{2}\mathbf{a}_i \end{cases}$$

- Then, use those to perform the full step:

$$\begin{cases} \mathbf{x}_{i+1} = \mathbf{x}_i + \Delta t \mathbf{v}_{i+\frac{1}{2}} \\ \mathbf{v}_{i+1} = \mathbf{v}_i + \Delta t \mathbf{a}_{i+\frac{1}{2}} \end{cases}$$



1st Order Explicit Euler Step